



PJMatch: Dokumentacja techniczna

Autorzy: Jakub Draszkiewicz, Nicolas Grdeń, Piotr Haber, Michał Jacewicz,
Albert Krysiak, Michał Liberacki, Piotrek Marwitz, Hubert Rotkiewicz
Maciej Truchanowski, Aleksander Tomaszewski, Kornel Wasilewski,
Oskar Wędołowski

25 czerwca 2026

Spis treści

1	Stack technologiczny i licencje	3
2	Opis problemu	3
3	Opis rozwiązań	3
3.1	Dataset	3
3.2	Text-to-PJM	4
3.2.1	Moduł NLP	4
3.2.2	FastAPI	4
3.2.3	Aplikacja wizualna	5
3.3	PJM-to-text	5
3.3.1	CSLR (Continuous Sign Language Recognition)	5
3.3.2	ISLR (Isolated Sign Language Recognition)	7
3.3.3	Korekta zdań z wykorzystaniem LLM	8
4	Opis problemów w trakcie realizacji projektu	10
4.1	Dataset	10
4.1.1	Akwizycja danych	10
4.1.2	Inne	10
4.2	Text-to-PJM	11
4.2.1	Animacje	11
4.2.2	Silnik Unreal Engine 5	11
4.2.3	Laboratorium	11
4.2.4	Organizacja plików	11
4.2.5	Backend	12
4.3	PJM-to-text	12
4.3.1	Praca nad modelem sieci	12
4.3.2	Wyniki modelu	12
5	Perspektywa dalszego rozwoju projektu	12

1 Stack technologiczny i licencje

- **Dataset:** Python¹, Pandas², imageio-ffmpeg³, React Native (Expo)³, Stanza⁴
- **PJM-to-text:** Python¹, PyTorch², MediaPipe⁴, MMPose⁴
- **Text-to-PJM:** Python¹, SpaCy-Stanza⁴, FastAPI³, Uvicorn², Unreal Engine 5⁵, VaRest³

2 Opis problemu

Głównym problemem, na który odpowiada projekt, jest bariera komunikacyjna pomiędzy osobami posługującymi się Polskim Językiem Migowym (PJM) a osobami słyszącymi. Przewycięzenie tej bariery wymaga stworzenia skutecznego systemu dwukierunkowego tłumaczenia w czasie rzeczywistym. Realizacja tak złożonego systemu wiąże się z koniecznością rozwiązania problemów na dwóch głównych płaszczyznach: budowy odpowiedniego zbioru danych (datasetu) oraz opracowania modułów translacyjnych.

3 Opis rozwiązań

3.1 Dataset

Projekt zakładał stworzenie zbioru danych składającego się z 5000 nagrań zdań Polskiego Języka Migowego. Aby to osiągnąć zaimplementowano szerego rozwiązań, które pomogły w realizacji tego założenia:

- **Własna nagrywarka:** Stworzenie od zera dedykowanej aplikacji mobilnej. Rozwiązało to problem zapisu w galerii oraz pozwoliło na automatyczne tworzenie plików `.json` do każdego nagrania.
- **Piloty Bluetooth:** Aplikacja została zintegrowana z bezprzewodowymi pilotami. Rozwiązało to problem trzęsienia się telefonu na statywie.
- **Kolejka nagrań w tle:** Wdrożenie systemu kolejkowania wysyłania plików. Dzięki temu wolny internet stał się mniejszym problemem ponieważ można nagrywać kolejne zdania, podczas gdy poprzednie wysyłają się w tle.
- **Automatyzacja procesu:** Dodanie funkcji automatycznego przełączania na kolejne zdanie po zakończeniu nagrania. Po ustawieniu w telefonie odpowiedniej osoby i pierwszego nagrywanego zdania użytkownik używa tylko pilota do nagrywania, bez konieczności wpisywania czegokolwiek w aplikacji.
- **Walidator:** Stworzenie skryptów serwerowych, które dbają o porządek w bazie (czyszczenie dubli, usuwanie ścieżek audio, weryfikacja plików `.json` z Excelem).
- **Skrypt statystyczny:** Stworzenie skryptu do tworzenia statystyk występowania glossów w zdaniach z datasetu wraz z kontekstami występowania w zdaniach.

¹Python Software Foundation License (PSFL)

²BSD 3-Clause License

³MIT License

⁴Apache License Version 2.0

⁵Unreal Engine End User License Agreement (EULA)

- **Skrypt filtrowania zdań:** Stworzenie skryptu odsiewających zdania, które posiadają znaki spoza N najczęściej występujących znaków w datasetcie.
- **Aplikacja do śledzenia ilości zdań:** Stworzenie aplikacji, która pokazuje ile zdań zostało nagrane i jakie osoby nagrały konkretne zdanie.

3.2 Text-to-PJM

Projekt zakładał stworzenie modułu tłumaczącego tekst na PJM. W tym celu stworzono aplikację w środowisku Unreal Engine 5, która wyświetla awatara migającego zadane zdanie, które wcześniej zostało odpowiednio przerworzone i przetłumaczone.

3.2.1 Moduł NLP

Backend modułu Text-to-PJM odpowiada za przetwarzanie tekstu wejściowego w języku polskim na uporządkowaną reprezentację glos, która następnie jest wykorzystywana przez aplikację wizualną do odtworzenia odpowiednich animacji PJM.

Pierwszym etapem działania systemu jest analiza tekstu wejściowego. Użytkownik podaje zdanie lub zestaw zdań w języku polskim, które trafiają do modułu odpowiedzialnego za lematyzację oraz uproszczoną transformację składniową. Do analizy językowej wykorzystano bibliotekę `spacy_stanza`, która pozwala na przetwarzanie języka polskiego z uwzględnieniem takich informacji jak lemat wyrazu, część mowy, zależności składniowe oraz cechy morfologiczne.

Głównym zadaniem modułu lematyzacji jest sprowadzenie odmienionych form wyrazów do ich form podstawowych. Przykładowo słowa takie jak *mam*, *miałem* lub *miała* mogą zostać sprowadzone do lematu *mieć*. Dzięki temu system może łatwiej dopasować słowa z tekstu wejściowego do dostępnych animacji, których nazwy są przechowywane w formie bazowej. Jest to szczególnie istotne w języku polskim, ponieważ jedno słowo może występować w wielu różnych formach gramatycznych.

Oprócz samej lematyzacji backend wykonuje dodatkowe operacje przygotowujące tekst do reprezentacji w PJM. System pomija część elementów, które nie są bezpośrednio reprezentowane w docelowym zapisie głosowym, a także zmienia kolejność wyrazów. Moduł został również rozszerzony o obsługę zdań pytających, przeczeń, czasowników posiłkowych, wyrażen czasowych oraz podział zdań na klauzule.

Wygenerowana reprezentacja tekstu uwzględnia formy podstawowe wyrazów, kolejność glos oraz dodatkowe oznaczenia zgodne z przyjętym sposobem prezentowania treści w Polskim Języku Migowym.

3.2.2 FastAPI

Drugim elementem backendu jest aplikacja API przygotowana z wykorzystaniem frameworka **FastAPI**. Jej zadaniem jest udostępnienie funkcji przetwarzania tekstu innym częściom systemu, przede wszystkim aplikacji wizualnej stworzonej w Unreal Engine.

Aplikacja API udostępnia endpoint, do którego przesyłany jest tekst wpisany przez użytkownika. Po odebraniu żądania backend przekazuje tekst do modułu NLP, a następnie zwraca wynik w formacie JSON. Odpowiedź zawiera listę glosów oraz elementów specjalnych, które mogą zostać wykorzystane przez aplikację wizualną do wyszukania i odtworzenia odpowiednich animacji.

Po wygenerowaniu glosów backend sprawdza, czy dla każdego z nich istnieje odpowiadająca animacja. Lista dostępnych animacji jest wczytywana z pliku, w którym zapisano nazwy animacji znajdujących się w projekcie. Jeżeli dana glosa posiada odpowiadającą jej animację, zostaje przekazana do aplikacji wizualnej jako nazwa animacji do odtworzenia. W przypadku braku gotowej animacji system realizuje literowanie danego słowa.

FastAPI umożliwia komunikację z aplikacją wizualną przez żądania HTTP. Dzięki temu część odpowiedzialna za przetwarzanie tekstu jest oddzielona od projektu w Unreal Engine i może być rozwijana niezależnie od front-endu.

3.2.3 Aplikacja wizualna

Główną część aplikacji stanowi środowisko Unreal Engine 5, w którym realizowana jest wizualizacja języka migowego przy użyciu modelu awatara MetaHuman.

Na początkowym etapie projektu animacje były przygotowywane z wykorzystaniem programu Blender oraz dodatku OpenMocap. Rozwiązanie to pozwalało na automatyczne generowanie animacji na podstawie nagrań z jednej, bądź dwóch kamer. Niestety jednak występowały w tym rozwiązaniu liczne problemy związane na przykład z kompatybilnością szkieletów lub poprawnym eksportem animacji do formatu FBX. Dodatkowo pojawiały się trudności podczas przenoszenia animacji do środowiska Unreal Engine 5 oraz ich retargetowania na model MetaHuman. Z tego względu zdecydowano się na zmianę sposobu generowania animacji.

W dalszej części projektu wykorzystano Laboratorium Zanurzonej Wizualizacji Przestrzennej (LZWP), wyposażone w system śledzenia ruchu dłoni (finger tracking). Pozwoliło to na dokładniejsze odwzorowanie ruchów palców i dłoni, które stanowią kluczowy element języka migowego. Zarejestrowane animacje były bezpośrednio importowane do środowiska Unreal Engine 5 i przypisywane do modelu MetaHuman.

Proces działania systemu rozpoczyna się od przekształcenia zdania tekstowego na sekwencję glosów odpowiadających poszczególnym znakom języka migowego. Każdy glos jest powiązany z odpowiednią animacją przechowywaną w aplikacji.

Komunikacja pomiędzy modułem tłumaczenia a aplikacją została zrealizowana z wykorzystaniem biblioteki VaRest oraz frameworka FastAPI. Moduł tłumaczenia zwraca odpowiedź w postaci formatu JSON zawierającego listę nazw animacji odpowiadających poszczególnym znakom.

Po stronie Unreal Engine 5 przygotowano strukturę Blueprintów odpowiedzialną za obsługę interfejsu użytkownika, wysyłanie zapytań HTTP oraz odbieranie odpowiedzi z serwera FastAPI. Otrzymane dane są następnie przekazywane do systemu odpowiedzialnego za odtwarzanie animacji.

W celu zarządzania sekwencjami znaków zaimplementowano komponent **PJMPlayer**. Komponent ten odpowiada za tworzenie kolejki animacji, ich sekwencyjne odtwarzanie oraz zarządzanie czasem przejść pomiędzy kolejnymi znakami. Dzięki temu możliwe jest automatyczne prezentowanie całych zdań bez konieczności ręcznego uruchamiania poszczególnych animacji.

Do odtwarzania znaków wykorzystano mechanizmy dostępne w Unreal Engine 5, takie jak Animation Montage, Slot Animations oraz Inertialization. Pozwoliło to na uzyskanie bardziej naturalnych i płynnych przejść pomiędzy kolejnymi znakami języka migowego.

Całość została zaprojektowana w sposób modułowy, dzięki czemu możliwe jest dalsze rozszerzanie bazy znaków, rozbudowa modułu tłumaczenia oraz implementacja kolejnych funkcjonalności bez konieczności modyfikowania głównej architektury aplikacji.

3.3 PJM-to-text

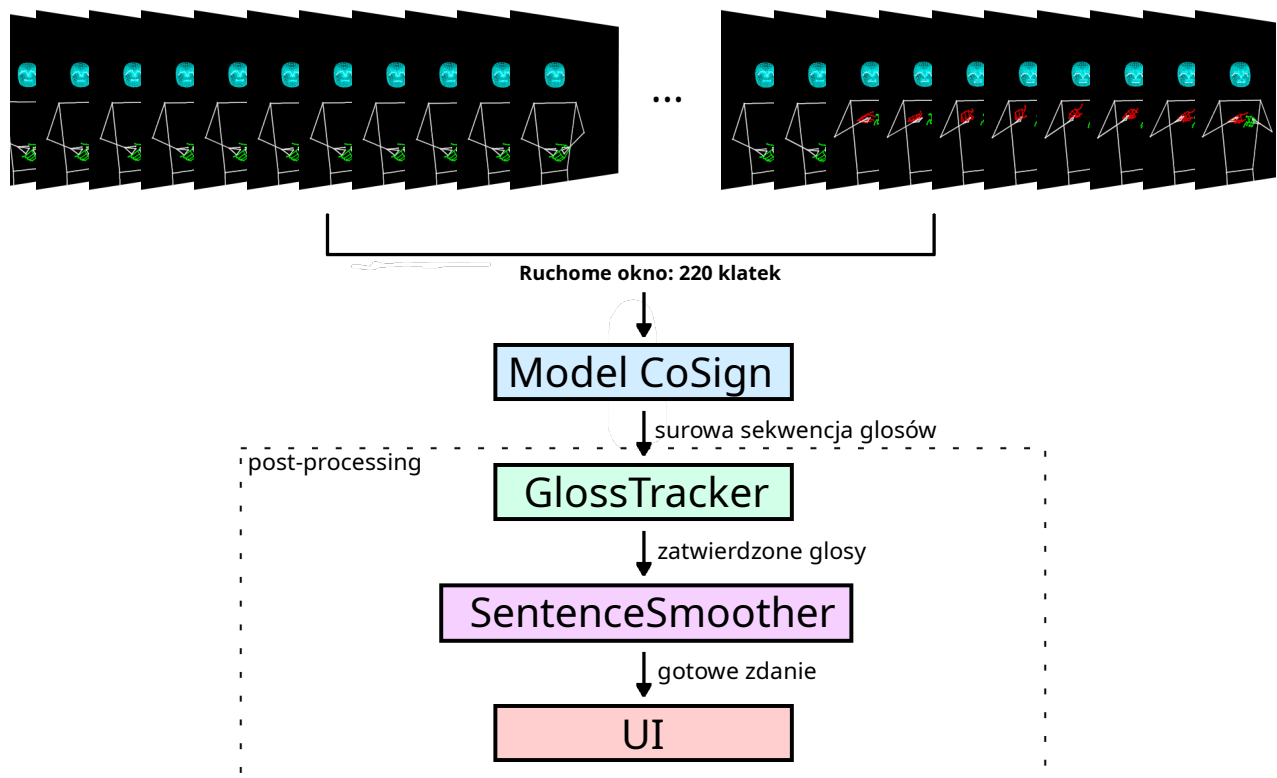
3.3.1 CSLR (Continuous Sign Language Recognition)

Projekt przewidywał utworzenie modułu tłumaczenia ciągłego języka migowego. Oznacza to tłumaczenie bez podziału na pojedyncze słowa, z całych sekwencji glosów interpretowanych przez sieć neuronową. Jako architekturę sieci wybrano model CoSign-1s zaproponowany w pracy [3]. Przeprowadzono trening na zgromadzonym zbiorze uczącym (opisanym w sekcji 3.1). Odtworzony model porównano z oryginalnymi wynikami z publikacji [3].

1. Pheonix-14T - dataset do CSLR dla języka niemieckiego, zawierający 14 000 nagrań wideo. Zbiór ten jest powszechnie używany jako benchmark w dziedzinie rozpoznawania języka migowego.
2. PJM signer independant - w zbiorze Train i test pojawiają się te same zdania ale różni migający
3. PJM sentence independant - w zbiorze Train i test pojawiają się różne zdania ale ci sami migający

Niskie wyniki modelu na datasetcie Pheonix-14T nie są zgodne z wynikami publikacji [3], co może wynikać z niejednoznacznego opisu implementacji w artykule. W związku z tym nie udało się zweryfikować poprawności odtworzonego modelu. WER na datasetcie PJM wynika z overfittingu modelu na zdaniach, w datasetcie nie zawarto wystarczającej liczby glossów użytych w różnych kontekstach, aby model mógł je poprawnie rozróżniać. Skutkuje to niskim WER na zdaniach ze zbioru uczącego - natomiast w przypadku gdy migane zdanie nie znajduje się w zbiorze uczącym, model nie jest w stanie poprawnie rozpoznać pojedynczych glosów.

Ze względu na fakt, że architektura CoSign-1s z założenia analizuje całe nagrania wideo i nie przewiduje tłumaczenia "w locie" (*ang. online translation*), na potrzeby niniejszego projektu stworzono system wykorzystujący technikę ruchomego okna (*ang. sliding window*).



Rysunek 1: Schemat potoku przetwarzania CSLR wykorzystujący architekturę CoSign-1s oraz autorskie moduły post-processingu.

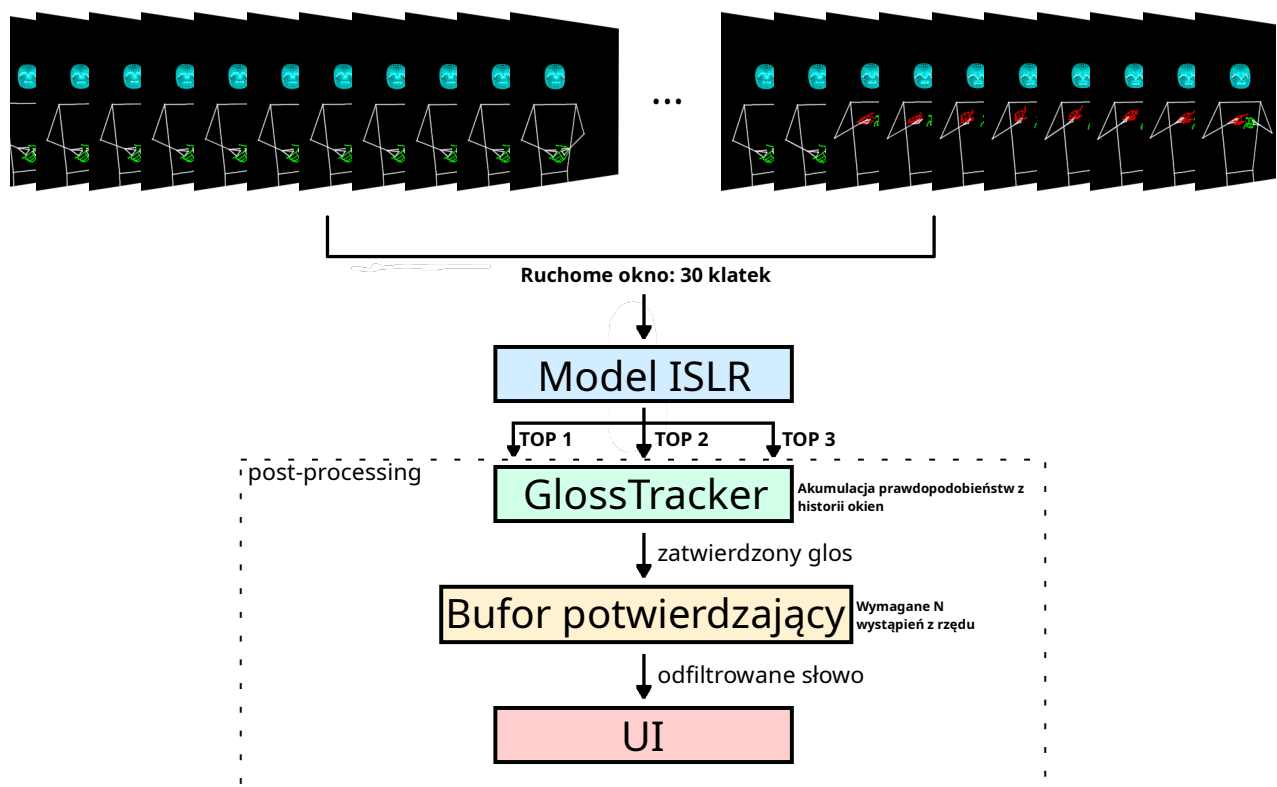
Jak przedstawiono na Rysunku 1, strumień wideo rejestrowany przez kamerę jest analizowany klatka po klatce z wykorzystaniem biblioteki MediaPipe [6], która ekstrahuje współrzędne punktów kluczowych twarzy, dłoni oraz postawy ciała. Zgromadzone dane przestrzenne trafiają do bufora o stałej długości 220 klatek, który przesuwa się w czasie o zdefiniowany krok (tzw. *stride*) równy 15 klatek. Dzięki temu model neuronowy może na bieżąco ewaluować ostatni fragment ruchu użytkownika, umożliwiając działanie aplikacji w czasie rzeczywistym.

W trybie CSLR wyekstrahowane okno danych jest przekazywane do sieci neuronowej, która przy użyciu algorytmu dekodowania CTC (*ang. Connectionist Temporal Classification* [1]) generuje surową

sekwencję głosów. Aby zniwelować błędy wynikające z fragmentacji predykcji dla nakładających się okien, zaprojektowano dwa autorskie moduły filtrujące:

- **GlossTracker** – moduł pełniący funkcję systemu konsensusu (*ang. hard voting*). Śledzi on predykcje z historii ostatnich przesunięć okna. Głos zostaje zatwierdzony tylko wtedy, gdy pojawi się minimalną, wymaganą liczbę razy, co skutecznie filtruje jednoklatkowe artefakty i halucynacje sieci.
- **SentenceSmoother** – algorytm wygładzania zdań, który analizuje podobieństwo (bazujące na części wspólnej i sumie zbiorów słów) nakładających się na siebie predykcji. Łączy on powtarzające się sekwencje w płynne, stabilne zdania, ignorując pojedyncze słowa, które nie utrzymały się w wynikach wystarczająco długo.

3.3.2 ISLR (Isolated Sign Language Recognition)



Rysunek 2: Schemat potoku przetwarzania ISLR wykorzystujący uczenie zespołowe.

Oprócz tłumaczenia ciągłego, zaimplementowano dedykowany potok przetwarzania (Rysunek 2) dla wyizolowanych znaków języka migowego. Algorytm również opiera się na mechanizmie ruchomego okna, jednak ze względu na specyfikę pojedynczych znaków, szerokość okna zredukowano do 30 klatek. Model w tym trybie został wyuczony na pojedynczych głosach wyekstrahowanych ze zbioru uczącego (sekcja 3.1).

Wewnętrzna architektura klasyfikatora izolowanych znaków bazuje na komponencie ST-GCN (*ang. Spatial-Temporal Graph Convolutional Network*) [7]. Implementacja tego modułu została oparta na otwartoźródłowym repozytorium [2] (implementacja ta została również użyta przy odtwarzaniu modelu CoSign-1s dla CSLR). Komponent ten odpowiada za ekstrakcję głębokich cech z sekwencji przestrzennych grafów (współrzędnych punktów kluczowych dłoni, twarzy i postawy). Wyekstrahowana przez moduł grafowy cechy - tensor o wymiarach $B \times 1024 \times T$ (gdzie B to rozmiar wsadu (*ang. batch*), a T oznacza liczbę klatek w oknie) jest następnie poddawana operacji redukcji wymiaru czasowego przy

użyciu mechanizmu uwagi (*ang. Attentive Pooling*). Mechanizm ten agreguje cechy wzdłuż osi czasu, ucząc się przypisywać wyższe wagi klatkom niosącym najważniejsze informacje semantyczne i pomijając klatki nieistotne. Uzyskany w ten sposób wektor cech o stałej długości 1024 trafia do w pełni połączonej warstwy liniowej (*ang. linear classification head*), która mapuje go na przestrzeń logitów odpowiadającą liczbie rozpoznawanych klas.

Metryka	Wynik
Top-1 Accuracy	84.41%
Top-3 Accuracy	95.00%
Top-5 Accuracy	96.39%
Macro recall	77.56%
Mean confidence correct	0.9151
Mean confidence incorrect	0.6525

Ze względu na niedostosowanie zbioru danych do zadania CSLR, zdecydowano się podzielić go na pojedyncze glosy, co umożliwiło trenowanie modelu ISLR. W ten sposób podzielono 3649 nagrań. Wprowadzono dodatkowo glos „BLANK”, wykorzystywany do wykrywania pozycji neutralnej. Poprawiło to wyniki modelu, a w praktycznym zastosowaniu skuteczniej zapobiega błędnej klasyfikacji w sytuacji, gdy osoba migająca nie wykonuje żadnego znaku, w porównaniu do samego zastosowania statycznego progu odcięcia (*threshold*) podczas inferencji. W przypadku ISLR zbiór danych podzielono na dwie części: uczącą i testową, przy czym zadbano o to, aby znaki obecne w zbiorze testowym nie występowały w tym samym kontekście zdaniowym co w zbiorze uczącym. Oznacza to, że model musiał nauczyć się rozpoznawać pojedyncze glosy niezależnie od kontekstu zdaniowego. Szczegółowe wyniki klasyfikacji przedstawiono w tabeli 1.

Ponieważ klasyczne modele ISLR są silnie wrażliwe na szum przejściowy (chaotyczny ruch rąk użytkownika pomiędzy właściwymi znakami), zastosowano alternatywne podejście do analizy prawdopodobieństw z warstwy Softmax. Zamiast wybierać wyłącznie najbardziej prawdopodobną klasę dla danego okna, system ekstrahuje trzy najlepsze predykcje i wykorzystuje technikę czasowego uczenia zespołowego (*ang. Temporal Ensembling* [5] / *Soft Voting* [4]).

GlossTracker w trybie ISLR sumuje ułamkowe pewności (prawdopodobieństwa) dla każdego znaku na przestrzeni 15 ostatnich okien czasowych (co przy zdefiniowanym kroku odpowiada pamięci wstecznej o długości 2,5 sekundy). Jeśli dany znak przekroczy zdefiniowany próg skumulowanego wyniku, zostaje natychmiast uznany za poprawny.

Podejście to skutecznie rozwiązuje problem szybkich znaków (np. krótkich zaimków takich jak „JA” czy „TY”), które trwają zbyt krótko, aby w pojedynczej klatce osiągnąć rygorystyczny próg pewności wymagany w podejściu *hard voting*. Ostateczna predykcja trafia do bufora potwierdzającego. Moduł ten eliminuje problem migotania interfejsu graficznego, wymagając stabilności decyzji sieci przez określoną liczbę iteracji przed zaprezentowaniem wyniku użytkownikowi. Ponadto system automatycznie czyści interfejs po zakończeniu ruchu (detekcja i zatwierdzenie klasy *blank*).

3.3.3 Korekta zdań z wykorzystaniem LLM

Zarówno potok CSLR, jak i ISLR zwracają wynik w postaci surowej sekwencji głosów czyli nieodmienionych form podstawowych poszczególnych znaków (np. MAMA KUPIĆ CHLEB). Sekwencje takie nie stanowią poprawnych zdań w języku polskim, ponieważ pozbawione są, właściwego szyku. W celu zaprezentowania użytkownikowi naturalnego tłumaczenia, na końcu potoku przetwarzania umieszczono dodatkowy moduł korekty, oparty na dużym modelu językowym Bielik, uruchamianym lokalnie za pośrednictwem serwera Ollama.

Wybór rozwiązania działającego lokalnie zamiast komercyjnego API w chmurze, podyktowany był wymogiem ochrony prywatności oraz brakiem zależności od połączenia sieciowego i kosztów zapytań. Model Bielik wybrano ze względu na jego specjalizację w języku polskim, co jest kluczowe dla poprawnej odmiany fleksyjnej generowanych zdań.

Architektura komunikacji. Moduł LLM zaimplementowano jako osobny wątek roboczy (klasa *BielikWorker*, dziedzicząca po *QThread*). Wydzielenie zapytań sieciowych do oddzielnego wątku zapobiega blokowaniu interfejsu graficznego oraz głównej pętli inferencji w czasie oczekiwania na odpowiedź modelu. Przepływ danych zrealizowano w oparciu o mechanizm sygnałów i slotów biblioteki Qt: moduł rozpoznający znaki (*AIWorker*) po zatwierdzeniu każdego głosu emituje sygnał *prediction_ready*, który trafia do bufora modelu LLM. Gotowe, przetłumaczone zdanie jest następnie zwracane sygnałem *word_received* i prezentowane w interfejsie użytkownika.

Mechanizm buforowania i wyzwalania. Aby uniknąć wysyłania zapytania po każdym pojedynczym znaku, zastosowano mechanizm buforowania z opóźnieniem (*ang. debounce*). Rozpoznane głosy gromadzone są w buforze (z pominięciem wartości pustych oraz klasy **blank**), a każdorazowo odświeżany jest znacznik czasu ostatniego znaku. Wątek roboczy cyklicznie sprawdza stan bufora i wyzwala zapytanie dopiero wtedy, gdy od momentu zarejestrowania ostatniego głosu upłynął zdefiniowany czas bezczynności (3 sekundy). Stan taki interpretowany jest jako zakończenie miganego zdania przez użytkownika. Zgromadzone głosy są wówczas scalane w jeden ciąg, bufor jest czyszczony, a do serwera Ollama wysyłane jest pojedyncze żądanie HTTP.

Format zapytania. Komunikacja odbywa się poprzez żądanie typu POST kierowane do lokalnego punktu końcowego API Ollama (<http://localhost:11434/api/generate>). Ładunek żądania w formacie JSON zawiera nazwę modelu ("bielik"), flagę **stream** ustawioną na **false** (odpowiedź zwracana jest w całości, a nie strumieniowo) oraz polecenie czyli prompt. Polecenie zaprojektowano w taki sposób, aby ograniczyć halucynacje modelu i wymusić wierność tłumaczenia względem wejściowych głosów. Zdefiniowano w nim następujące rygorystyczne zasady:

- zakaz dodawania słów, których nie ma w głosach wejściowych;
- ignorowanie bezpośrednich powtórzeń (np. MAMA MAMA KUPIĆ traktowane jak MAMA KUPIĆ);
- zwrócenie wyłącznie przetłumaczonego zdania, bez komentarzy, wstępów czy cudzysłówów;
- stosowanie poprawnej odmiany przez przypadki oraz odpowiednich czasów gramatycznych.

Odpowiedź modelu (pole **response** w zwróconym obiekcie JSON) jest odczytywana, oczyszczana z białych znaków i przekazywana do warstwy prezentacji. Ewentualne błędy komunikacji (np. niedziałający serwer Ollama) są przechwytywane i raportowane bez przerywania działania aplikacji.

Wdrożenie modelu. Model dystrybuowany jest w postaci pliku w formacie GGUF (np. *Bielik-Minitron-7B-v3.0-Instruct-Q4_K_M.gguf*). W celu udostępnienia go przez serwer Ollama definiuje się plik konfiguracyjny *Modelfile*, wskazujący na lokalny plik z wagami, a następnie rejestruje model pod nazwą **bielik** (wymaganą przez kod aplikacji). Procedurę przedstawiono poniżej.

Listing 1: Rejestracja i uruchomienie modelu Bielik w serwerze Ollama.

```
1 # Zawartosc pliku Modelfile (umieszczonego w katalogu z plikiem .gguf):
2 # FROM ./Bielik-Minitron-7B-v3.0-Instruct-Q4_K_M.gguf
3
4 # Rejestracja modelu pod nazwa "bielik":
```

```
5 ollama create bielik -f Modelfile
6
7 # Uruchomienie serwera modelu (pozostaje aktywny w tle):
8 ollama run bielik
```

Wyniki ewaluacji W celu oceny skuteczności modułu korekty zdań przeprowadzono eksperyment na zbiorze 1300 próbek wygenerowanych przez przetrenowany model CSLR. Wykorzystany model rozpoznawania języka migowego osiągał wcześniej wartość WER równą 55%, co oznaczało stosunkowo dużą liczbę błędów w generowanych sekwencjach głosów. Pomimo niedoskonałości danych wejściowych, integracja z modelem językowym Bielik pozwoliła uzyskać wartość BLEU-1 równą 48.82 oraz BLEU-2 równą 33.10. Otrzymane wyniki wskazują, że model językowy był w stanie częściowo skorygować błędy występujące w sekwencjach głosów oraz wygenerować zdania bliższe naturalnemu językowi polskiemu. Jednocześnie niższa wartość BLEU-2 sugeruje, że największe trudności dotyczyły zachowania poprawnych zależności pomiędzy kolejnymi wyrazami, co wynikało przede wszystkim z błędów popełnianych przez moduł rozpoznawania PJM.

4 Opis problemów w trakcie realizacji projektu

4.1 Dataset

4.1.1 Akwizycja danych

- Zapis do pamięci telefonu/galerii łamie zasady prywatności.
- Drgania obrazu niszczące jakość datasetu.
- Przestoje w nagrywaniu wywołane słabym połączeniem sieciowym i oczekiwaniem na wysłanie pliku.
- Zbyt długie przerwy między nagraniami wywołane wprowadzaniem danych takich jak numer aktualnego nagrania w aplikacji.
- Długi czas oczekiwania na tłumaczenia spowodowany brakiem czasu wolontariusza.

W celu rozwiązania problemów związanych z pozyskiwaniem danych stworzono autorską aplikację mobilną, która bezpośrednio łączy się z serwerem, a następnie (bez zapisu w wewnętrznej pamięci telefonu) przesyła na niego plik wideo. Aby zminimalizować ryzyko występowania nieporządanych drgań kamery zastosowano piloty na podczerwień.

W trakcie realizacji nagrań (zwłaszcza w okresie początkowym) mocno optymalizowano aplikację pod kątem zmniejszenia czasu oczekiwania na przesył danych (np. kolejnowanie).

4.1.2 Inne

- Niektórzy wolontariusze się nie zjawiali lub odwoływali nagrywki.
- Nikt z zespołu nie znał języka migowego, co utrudniało weryfikację wyników modelu i poprawki w anotacji datasetu. Rozwiązano to przez współpracę z wolontariuszami i samodzielną nauką języka migowego.

W czasie nagrań jak i w trakcie procesu anotowania zbioru umiejętność rozpoznawania znaków była konieczna dla weryfikacji poprawności nagrań. Z tego powodu każdy członek zespołu pracujący przy zbieraniu oraz opisywaniu danych uczył się występujących w datasecie znaków.

4.2 Text-to-PJM

4.2.1 Animacje

- Niedokładność animacji generowanych z pluginu OpenMocap.
- Niekompatybilność szkieletów blendera i Unreal Engine 5 (błędy retargetowania animacji).
- Problemy z przenoszeniem animacji z blendera do Unreal Engine 5.
- Problemy z płynnością przejść animacji.
- Ucinanie końców animacji przez łączenie animacji.
- Powrót awatara do pozycji neutralnej po zakończeniu każdej z animacji (przy małym czasie przejścia).

W celu rozwiązania problemów związanych z animacjami zrezygnowano z wykorzystania OpenMocap jako głównego źródła nagrywania i przeniesiono proces nagrywania animacji do Laboratorium Zanurzonej Wizualizacji Przestrzennej. Dodatkowo dobrano odpowiednie parametry mieszania animacji oraz mechanizmu Inertialization w UE5, co pozwoliło uzyskać płynniejsze przejścia pomiędzy poszczególnymi animacjami.

4.2.2 Silnik Unreal Engine 5

- Problemy z awatarem MetaHuman.
- Częste awarie Unreal Engine 5.
- Problemy z importem animacji do Unreal Engine 5 (brakujące kości, niekompatybilne szkieletu).

Problemy związane z importem animacji zostały w dużej mierze wyeliminowane dzięki zmianie sposobu pozyskiwania animacji i bezpośredniemu wykorzystaniu nagrań z laboratorium LZWP. Część problemów związanych ze stabilnością silnika UE5 nadal występuje i wynika najprawdopodobniej z ograniczeń środowiska.

4.2.3 Laboratorium

- Niedokładność śledzenia ruchu palców w systemie finger tracking.
- Poruszanie się awatara poprzez błędy w systemie śledzenia ruchu palców (inverse kinematics w Unreal Engine 5 nie działało najlepiej we współpracy z ruchem dłoni).
- Potrzeby poprawiania animacji przez niedokładności.

W celu poprawy jakości animacji ustabilizowano szkielet, co pozwoliło ograniczyć niepożądane ruchy awatara w osi góra-dół. Dodatkowo przeprowadzono ręczne poprawki i dostrajanie animacji w celu uzyskania bardziej naturalnych ruchów dłoni i całego ciała.

4.2.4 Organizacja plików

- Zbyt duże pliki z projektu nie dające się przesyłać na repozytorium.

Problemy związane z rozmiarem projektu nie zostały wyeliminowane ze względu na ograniczenia platformy GitHub dotyczące wielkości przesyłanych plików. W związku z tym część dużych zasobów była przechowywana lokalnie przez każdego członka zespołu.

4.2.5 Backend

- Problemy z lematyzacją i transformacją składniową tekstu wejściowego.
- Trudności z obsługą zdań złożonych i podziałem tekstu na klauzule.
- Halucynacje modelu spacy_stanza.

4.3 PJM-to-text

4.3.1 Praca nad modelem sieci

- Wymagania przepustowości modelu estymującego pozę.
- Niejednoznaczny opis implementacji modelu cosign w artykule [3], co uniemożliwiło weryfikację implementacji.
- Debugowanie sieci neuronowej.
- Pretraining na datasetach LSFB, Pheonix-14T, nie przyniósł spodziewanych rezultatów; model nie miał lepszych wyników niż trening od zera.

Z powodu wysokich wymagań dla przepustowości estymatora pozycji ciała, dłoni oraz wyrazu twarzy należało odpowiednio dobrać jego model. Podczas gdy [3] wykorzystywał bibliotekę MMPose do ekstrakcji szkieletu, w niniejszym projekcie zdecydowano się na MediaPipe [6] ze względu na znacznie mniejsze wymagania obliczeniowe, kosztem pewnej dokładności estymacji pozy.

Ze względu na charakter sieci neuronowych zdecydowano się na szereg rozwiązań pozwalających na głębsze zrozumienie błędów występujących w trakcie realizacji projektu. Eksperymentalny dobór parametrów, pokrycie kluczowych segmentów modelu testami jednostkowymi oraz reprezentacja warstw sieci w postaci grafów pozwoliły na optymalną implementację rozwiązania.

4.3.2 Wyniki modelu

- Niepoprawny design datasetu do CSLR (za mało próbek głosów w różnych kontekstach, żeby nauczyć model je rozróżniać) czego skutkiem był overfitting modelu.
- Dataset składał się ze zdań, przez co nie był adekwatny do nauki modelu ISLR, który wymaga pojedynczych słów.
- Annotacje datasetów bywały niepoprawne, co zaniżało wyniki.

Aby poradzić sobie z zainstniałymi problemami bez konieczności zbierania nowego datasetu zdecydowano się na równoległe rozwijanie modułu ISLR. Z tego powodu należało ręcznie przejrzeć każde nagranie, a następnie odpowiednio je zaanotować, oznaczając początek i koniec każdego znaku w nim występującego. Pozwoliło to również wyeliminować istniejące już błędy, jeżeli takie wystąpiły.

5 Perspektywa dalszego rozwoju projektu

Dalszy rozwój projektu może przebiegać w kilku uzupełniających się kierunkach. Najbardziej bezpośrednim krokiem jest rozbudowa zbioru danych poprzez nagranie większej liczby próbek w kontrolowanych warunkach.

Równolegle można rozważyć udostępnienie aplikacji mobilnej umożliwiającej zdalną akwizycję danych, użytkownicy mogliby samodzielnie przysyłać nagrania bez konieczności angażowania członków zespołu. Docelowo do systemu mógłby zostać dodany moduł aplikacji edukacyjnej wzorowanej na platformach takich jak Duolingo, oferującej użytkownikowi informację zwrotną dotyczącą poprawności wykonania znaku. Analiza rozbieżności między gestem użytkownika a wzorcem oczekiwanym przez model pozwoliłaby na wskazanie konkretnych aspektów wymagających korekty, umożliwiając samodzielną naukę znaków zawartych w zbiorze treningowym.

Bibliografia

- [1] Alex Graves i in. „Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks”. W: 2006.
- [2] hazdzz. *STGCN: Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition (PyTorch Implementation)*. <https://github.com/hazdzz/STGCN>. 2020.
- [3] Peiqi Jiao i in. „CoSign: Exploring Co-occurrence Signals in Skeleton-based Continuous Sign Language Recognition”. W: *Nazwa czasopisma lub konferencji* 12.3 (2023), s. 45–56.
- [4] J. Kittler i in. „On combining classifiers”. W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.3 (1998), s. 226–239. DOI: 10.1109/34.667881.
- [5] Samuli Laine i Timo Aila. „Temporal ensembling for semi-supervised learning”. W: *arXiv preprint arXiv:1610.02242* (2016).
- [6] Camillo Lugaresi i in. „MediaPipe: A Framework for Building Perception Pipelines”. W: *CoRR* abs/1906.08172 (2019). arXiv: 1906.08172. URL: <http://arxiv.org/abs/1906.08172>.
- [7] Sijie Yan, Yuanjun Xiong i Dahua Lin. „Spatial temporal graph convolutional networks for skeleton-based action recognition”. W: *Thirty-second AAAI conference on artificial intelligence*. 2018.

Tabela 1: Wyniki klasyfikacji izolowanych głosów PJM

Gloss	Poprawne	Wszystkie	Dokładność	Najczęstsza pomyłka
NIE	0	1	0,0%	SŁYSZEĆ (1/1)
POMÓC-MI	0	1	0,0%	NOWY (1/1)
TO	0	5	0,0%	TY (2/5)
WIECZÓR	0	2	0,0%	NOC (2/2)
TERAZ	2	22	9,1%	DZISIAJ (20/22)
PAMIĘTAĆ	4	18	22,2%	POMYSŁ (11/18)
DLA	4	18	22,2%	TEN (4/18)
SŁYSZEĆ	4	16	25,0%	NIE-SŁYSZEĆ (9/16)
ROBIĆ	5	18	27,8%	CO-ROBIĆ (7/18)
NIE-MIEĆ	9	20	45,0%	NIE-CHCIEĆ (8/20)
ZOSTAĆ	7	15	46,7%	DUŻY (4/15)
WY	10	21	47,6%	MY (8/21)
SIOSTRA	17	35	48,6%	RODZEŃSTWO (8/35)
ON	12	24	50,0%	TY (8/24)
WIDZIEĆ	1	2	50,0%	WIEDZIEĆ (1/2)
ŚRODA	1	2	50,0%	DOBRY (1/2)
TEN	31	56	55,4%	TA (6/56)
ZIMNY	33	59	55,9%	MAMA (6/59)
IŚĆ	36	59	61,0%	PÓJŚĆ (7/59)
JUŻ	12	19	63,2%	KUPIĆ (5/19)
RODZEŃSTWO	25	39	64,1%	BRAT (13/39)
JEŚĆ	22	34	64,7%	JEDZENIE (8/34)
POMYSŁ	2	3	66,7%	RANO (1/3)
URODZINY	2	3	66,7%	CHCIEĆ (1/3)
CO	13	19	68,4%	PARA (2/19)
SAMOCHÓD	42	58	72,4%	DZISIAJ (7/58)
ILE	28	38	73,7%	GDZIE (3/38)
KIEDY	14	19	73,7%	CO (1/19)
BYĆ	30	40	75,0%	CO (2/40)
WODA	3	4	75,0%	MAMA (1/4)
PRZEPRASZAM	3	4	75,0%	ZE-MNĄ (1/4)
GDZIE	43	56	76,8%	CHCIEĆ (5/56)
PRZED	17	22	77,3%	TO (1/22)
KAWA	42	54	77,8%	PRACA (10/54)
KLUCZ	14	18	77,8%	TY (2/18)
KOT	35	44	79,5%	DOBRY (2/44)
SPAĆ	43	54	79,6%	RANO (9/54)
HERBATA	46	56	82,1%	PONIEDZIAŁEK (2/56)
KTO	29	35	82,9%	KIEDY (2/35)
CZAS	30	36	83,3%	DUŻY (3/36)
WTOREK	15	18	83,3%	BARDZO (1/18)
MAŁY	15	18	83,3%	PRZED (1/18)
MĘŻCZYZNA	59	70	84,3%	TY (4/70)

Kontynuacja na następnej stronie

Tabela 1: Wyniki klasyfikacji izolowanych głosów PJM

Gloss	Poprawne	Wszystkie	Dokładność	Najczęstsza pomyłka
PIES	59	70	84,3%	MIEĆ (4/70)
DOBRY	49	58	84,5%	TWÓJ (5/58)
PIĆ	49	58	84,5%	DZIEŃ (2/58)
JAK	33	39	84,6%	MÓWIĆ (3/39)
PRACA	84	99	84,8%	PRACOWAĆ (12/99)
SMUTNY	17	20	85,0%	LUBIĆ (1/20)
WESOŁY	18	21	85,7%	NIE-CHCIEĆ (1/21)
MAMA	6	7	85,7%	ZIMNY (1/7)
PRZYJACIEL	18	21	85,7%	KOLEGA (2/21)
BABCIA	31	36	86,1%	STARY (2/36)
MÓWIĆ	32	37	86,5%	GORĄCY (2/37)
LUBIĆ	13	15	86,7%	NIE-LUBIĆ (1/15)
NOWY	101	116	87,1%	DZIEKOWAĆ (2/116)
ZDROWY	34	39	87,2%	URODZINY (2/39)
JUTRO	63	72	87,5%	WCZORAJ (6/72)
MIEĆ	59	67	88,1%	TWÓJ (2/67)
TELEFON	37	42	88,1%	WCZORAJ (2/42)
KOBIETA	46	52	88,5%	DOBRY (4/52)
WCZORAJ	85	96	88,5%	JUTRO (4/96)
NOC	32	36	88,9%	WIECZÓR (3/36)
TWÓJ	25	28	89,3%	DOBRY (2/28)
PONIEDZIAŁEK	17	19	89,5%	SPAĆ (2/19)
DZISIAJ	123	137	89,8%	SAMOCHÓD (6/137)
STARY	80	89	89,9%	BABCIA (5/89)
SZKOŁA	38	42	90,5%	WIEDZIEĆ (1/42)
KOLEGA	48	53	90,6%	PRZYJACIEL (2/53)
MUSIEĆ	31	34	91,2%	KLUCZ (1/34)
CZekać	21	23	91,3%	CO-ROBIĆ (1/23)
RANO	54	59	91,5%	KOT (1/59)
MY	33	36	91,7%	DLA-NAS (3/36)
MIGAĆ	67	73	91,8%	DOM (3/73)
KUPIĆ	70	76	92,1%	JUŻ (3/76)
ZMĘCZONY	35	38	92,1%	ZDROWY (1/38)
TY	145	156	92,9%	ON (3/156)
CHCIEĆ	67	72	93,1%	URODZINY (3/72)
BARDZO	127	136	93,4%	CO-ROBIĆ (1/136)
MÓJ	101	108	93,5%	MĘŻCZYŻNA (4/108)
KOCHAĆ	16	17	94,1%	NOWY (1/17)
DZIADEK	34	36	94,4%	KOBIETA (1/36)
TATA	35	37	94,6%	PIĆ (1/37)
KSIĄŻKA	18	19	94,7%	DLA (1/19)
DUŻY	74	78	94,9%	SAMOCHÓD (2/78)
DOM	75	79	94,9%	DUŻY (2/79)

Kontynuacja na następnej stronie

Tabela 1: Wyniki klasyfikacji izolowanych głosów PJM

Gloss	Poprawne	Wszystkie	Dokładność	Najczęstsza pomyłka
WAŻNY	19	20	95,0%	WIEDZIEĆ (1/20)
DZIEŃ	38	40	95,0%	blank (1/40)
CHORY	21	22	95,5%	KTÓRY (1/22)
JA	253	264	95,8%	SMUTNY (4/264)
DLACZEGO	54	56	96,4%	KUPIĆ (1/56)
UCZYĆ-SIĘ	39	40	97,5%	NOC (1/40)
PIENIĄDZ	39	40	97,5%	MAŁY (1/40)
ZAWSZE	46	47	97,9%	JAK (1/47)
blank	4988	4993	99,9%	SIOSTRA (1/4993)
MYŚLEĆ	5	5	100,0%	–
BRAT	18	18	100,0%	–
WIEDZIEĆ	23	23	100,0%	–
CZĘSTO	2	2	100,0%	–
DLA-CIEBIE	1	1	100,0%	–
DLA-MNIE	2	2	100,0%	–
NIE-CHCIEĆ	38	38	100,0%	–
GORĄCY	3	3	100,0%	–
RAZEM	1	1	100,0%	–
MI	1	1	100,0%	–
ROK	16	16	100,0%	–